

Módulo 4: Datos Sensibles y Secretos

T12: Secretos en Repos y Key Vault

Carlos Rodríguez Contreras · GitHub: [karrocon](#)

 *Claves de API en git, truffleHog y Azure Key Vault*

Objetivos

- Entender por qué secretos en el historial de git son una brecha permanente
- Usar truffleHog o git-secrets para detectar fugas en el historial
- Gestionar secretos correctamente con `.env` + `.gitignore`
- Introducción a gestores de secretos: HashiCorp Vault y Azure Key Vault

El problema: secretos en el código fuente

```
// ⚠ Esto aparece en TODOS los clones, forks y backups  
const JWT_SECRET = 'secret';  
const DB_PASSWORD = 'admin123';  
const API_KEY = 'sk-proj-abc123...';
```

¿Por qué es grave?

1. `git log -p --all -S 'API_KEY'` → visible para siempre en el historial
2. Borrar el archivo NO elimina el commit anterior
3. Forks y mirrors conservan el historial completo
4. Bots escanean GitHub en tiempo real buscando claves

Estadísticas reales

Dato	Valor
Secretos filtrados en GitHub (2023)	12.8 millones
Tiempo medio hasta detección	5 días
Tiempo medio hasta explotación	< 1 hora
Coste medio de un leak de API key	\$1.2M

💀 **Caso real:** Un desarrollador subió una clave AWS. En 15 minutos, un bot la encontró y lanzó instancias de minería de criptomonedas. Factura: \$14.000.

Herramientas de detección

Herramienta	Tipo	Cuándo usarla
git-secrets	Pre-commit hook	Bloquea commits con patrones de secretos
truffleHog	Scanner de historial	Busca alta entropía en todo el git log
gitleaks	CI/CD + local	SAST para secretos en pipelines
GitHub Secret Scanning	SaaS automático	Alerta cuando detecta tokens de proveedores

Escanear el repo actual con truffleHog

```
npx trufflehog filesystem . --no-verification
```

Con gitleaks

```
gitleaks detect --source . --report-path leaks.json
```

La solución: capas de protección

1. `.env` + `.gitignore` → No subir secretos al repo
2. `git-secrets` (pre-commit) → Bloquear si se intenta
3. CI scanner (`gitleaks`) → Detectar si se cuela
4. Secret manager (Key Vault) → No tener secretos en disco
5. Rotación automática → Limitar daño si se filtra

Gestores de secretos

Gestor	Tipo	Integración
Azure Key Vault	Cloud (Azure)	SDKs, Managed Identity, ARM
HashiCorp Vault	Self-hosted/Cloud	API REST, Agent sidecar
AWS Secrets Manager	Cloud (AWS)	IAM roles, Lambda
Doppler	SaaS	CLI, SDKs, integraciones CI/CD

```
// Ejemplo: leer secreto de Azure Key Vault
import { SecretClient } from '@azure/keyvault-secrets';
import { DefaultAzureCredential } from '@azure/identity';

const client = new SecretClient('https://my-vault.vault.azure.net', new DefaultAzureCredential());
const secret = await client.getSecret('jwt-secret');
const JWT_SECRET = secret.value!;
```

¿Qué hacer si ya subiste un secreto?

1. **Rotar inmediatamente** — genera una nueva clave/password
2. **Revocar la anterior** — invalida la clave filtrada en el proveedor
3. **No reescribir historial** — `git filter-branch` no elimina de forks/caches
4. **Documentar el incidente** — cuándo se filtró, qué acceso tenía, qué se hizo
5. **Añadir prevención** — git-secrets hook + CI scanner

⚠ `git filter-branch` / BFG Repo Cleaner eliminan del repo local, pero no de forks, caches de CI, mirrors ni backups.

Rotación automática de secretos

```
# Azure Key Vault – rotación automática de claves
resource keyVaultSecret 'Microsoft.KeyVault/vaults/secrets@2023-02-01' = {
  properties: {
    rotationPolicy: {
      lifetimeActions: [{
        trigger: { timeBeforeExpiry: 'P30D' } // 30 días antes de expirar
        action: { type: 'Notify' }
      }]
      attributes: { expiryTime: 'P90D' } // expira cada 90 días
    }
  }
}
```

Patrón para apps:

1. App lee secreto V2 de Key Vault
2. Rotación genera V3 y marca V2 como "next-to-expire"
3. App detecta nuevo secreto en siguiente lectura (o evento)
4. V2 se desactiva tras período de gracia

✓ La rotación automática elimina secretos "eternos" — si se filtra uno, solo vale unos días.

Lab T12: Detectando secretos en el historial de git

Objetivo: encontrar y eliminar secretos del historial de git

Setup: repo local del curso

1. Ejecuta `npx trufflehog filesystem .` sobre el directorio del proyecto
2. Observa si hay credenciales detectadas en archivos o historial
3. Crea un archivo `.env` con los secretos encontrados
4. Añade `.env` al `.gitignore` y verifica que no se trackea

 **Herramientas:** [truffleHog](#) · [git-secrets](#)